

XAI in code: A developer's hands-on approach

Let's now look at how XAI can be applied in early sepsis detection using Python libraries.

Sepsis is a life-threatening medical condition affecting over 1.7 million adults in the US annually, causing more than 270,000 deaths. Early detection is vital but challenging due to its subtle symptoms. AI and machine learning can identify sepsis risk early, but clinical adoption depends on trust and understanding of model predictions. Explainable AI (XAI), particularly using SHAP (SHapley Additive exPlanations), enables doctors to interpret these predictions.

Explainability is crucial in sepsis prediction because even though AI models trained on electronic health records can outperform traditional alerts, clinicians may not trust or act on alerts they don't understand. To gain their confidence, it's essential to clarify which features contributed to a high sepsis risk, ensure the insights are clinically relevant, and show how reliable the predictions are based on a patient's current and historical vitals.

There are several open source libraries for the interpretation and explanation of machine learning models, including LIME, ELI5, Anchor Explanations, InterpretML, DALEX, AIX360, and PyCaret, to name a few. Each of these libraries has something distinct and different to offer, from local model interpretability to high-level visualisation and debugging capabilities. But explaining all these tools is out of the domain of this article. For the sake of simplicity and ease of explanation, we will focus on demonstrating a straightforward example with the help of SHAP.

SHAP is an open source Python library for model interpretability, based on the concept of Shapley values from cooperative game theory. It explains the output of any machine learning model by computing the contribution of each feature to a specific prediction. SHAP was developed by Scott Lundberg, a researcher at Microsoft Research, and the methodology was introduced in a paper he wrote in 2017. Here's the Python program:

```
!pip install shap xgboost pandas matplotlib scikit-learn
--quiet
import shap
import xgboost
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Optional visual theme
plt.style.use('seaborn-whitegrid')

# -----
```

```
# Simulated Dataset
# -----
np.random.seed(42)
n = 1000

X = pd.DataFrame({
    'heart_rate': np.random.normal(90, 15, n),
    'resp_rate': np.random.normal(22, 5, n),
    'temp': np.random.normal(98.6, 1, n),
    'wbc': np.random.normal(11, 4, n),
    'lactate': np.random.normal(1.5, 0.8, n),
    'age': np.random.normal(65, 20, n),
})

# Target Label Logic
X['sepsis_risk'] = (
    (X['heart_rate'] > 100).astype(int) +
    (X['resp_rate'] > 24).astype(int) +
    (X['lactate'] > 2.5).astype(int) +
    (X['wbc'] > 12).astype(int)
)
y = (X['sepsis_risk'] >= 2).astype(int)
X.drop(columns='sepsis_risk', inplace=True)

# -----
# Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
model = xgboost.XGBClassifier(use_label_encoder=False, eval_
    metric='logloss')
model.fit(X_train, y_train)

# Accuracy
preds = model.predict(X_test)
print(f"Test Accuracy: {accuracy_score(y_test, preds):.2f}")

# -----
# Show Dataset Samples
print("\nPreview of Dataset Features:")
print(X.head())

print("\nFirst Test Sample (used for SHAP waterfall/force):")
print(X_test.iloc[0])

# SHAP Explainability
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)

# 1. Waterfall plot (local)
plt.figure(figsize=(10, 5))
print("\n[1] Waterfall Plot: Local SHAP Explanation for First
```